The AI 100 Final Project pushed me to use Generative AI as more than just a tool that produces code. I had to treat it like a thinking partner. What stood out most was how helpful it was in guiding me through problems instead of just giving answers. It kept pushing me to go deeper, especially when I was only looking at surface-level issues instead of the real conceptual mistakes behind them.

One of the biggest things I learned is that AI is really good at pointing out weak reasoning. There were times when I thought I understood what went wrong, but my explanations were shallow. The AI helped me see when I was focusing on symptoms, like strange errors or incorrect shapes, instead of the actual cause, such as problems with labels or misunderstandings about how parts of a model work.

A clear example of this was in Case 12. I intentionally broke the model by converting image data into strings. At first, I thought the issue had something to do with fonts not supporting the data. Looking back, that reasoning was completely off. The AI pushed me to rethink it, and I realized that neural networks do not interpret information the way humans do. They rely entirely on numbers and matrix operations. Once I understood that, the mistake made sense. That moment showed me how useful AI can be for correcting misunderstandings about how models actually work at a mathematical level.

Another example was in Case 9, when I removed a Flatten layer. My initial thought was that the Dense layer was just confused about dimensions. The AI helped me understand that each layer has a specific role. Convolutional layers handle spatial features, while dense layers work with flattened, global data. That explanation helped me visualize how data flows through the model step by step.

Breaking the model on purpose ended up being one of the most valuable parts of the project. It showed me how fragile deep learning systems can be and how precise everything needs to be for them to work correctly.

I also learned a lot about the importance of the data pipeline. A model is only as good as the relationship between its inputs and labels. When I ran into issues like mismatched data sizes, I realized you cannot just fix things by duplicating or cutting data. Doing that breaks the connection between the input and the correct output, which leads the model to learn meaningless patterns. The same idea appeared again in Case 12, where switching from numbers to strings completely broke the system because the training process depends on numerical data.

Another takeaway was how important the less visible settings are. Things like learning rate and activation functions might seem small, but they have a huge impact. I learned that normalization is critical because large input values can cause unstable training and high

loss. I also saw how using the wrong activation function can completely break a model. For example, using ReLU in the final layer instead of Softmax, like in Case 6, prevents the model from producing proper probabilities and causes it to behave randomly.

Finally, I realized that every layer in a neural network changes the shape of the data. Convolution and pooling layers transform or shrink the input, and if you are not careful, too much information can be lost before it reaches the end. Keeping track of these dimensions is essential.

Overall, this project helped me understand not just how to build models, but how they actually work at a deeper level.